Manual: run_deform_hits_config.py

`run_deform_hits_config.py` is the main DEFORM multihit runner. It reads an action CSV, creates one DEFORM database per hit, runs each database, prepares the next hit state, and exports files needed for post-processing and DOE studies.

Most users should configure the runner through `deform_config_gui.py`, then launch the generated JSON config. Advanced users can still run this file directly in Spyder or from the command line.

See the standalone flowchart: run_deform_hits_config_flowchart.md.

Inputs

- `DEFAULT_CONFIG` in `run_deform_hits_config.py`, used when no external config is passed.
- Optional external JSON config passed as `--config path\to\config.json`.
- A base DEFORM key file selected by `init_key`, `custom_key_name`, or the `key_files` template for the selected die.
- An action CSV. The configured `columns` map tells the runner which CSV columns contain offset, rotation, position, forging temperature, speed, and depth.

Important config choices

- `num_hits`: integer number of hits to run, or `"all"` to use every action row.
- `die_motion_mode`: `"symmetric"` or `"single_top_die"`.
- `apply_action_file_rules`: when true, action filenames containing `"bdm"` force symmetric both-dies-moving behavior.
- `die_motion_axis`: `"z"` for the original vertical die setup or `"y"` for dies that close along the Y axis.
- `height`: clearance after the die is seated flush to the workpiece.
- `time_step` and `save_step_size`: DEFORM solve controls.
- `stock_diameter`: supported GUI choices are `0.625` and `1` inch.
- `position_dies_before_first_hit`: enables hit-0 die seating from a general key.
- `position_bottom_die_before_first_hit`: seats the bottom die on hit 0 when the run uses both-dies-moving behavior.
- `change_material`: enables programmatic material replacement.
- `use_jmak`: controls DEFORM simulation modes. True forces `D H G`; false forces `D H`.
- `mean_grain_size_um`: initializes the JMAK grain state from supplier grain size data when `use_jmak` is true.
- `export_post_hit_keys`, `export_coordinate_logs`, `export_load_stroke`: control output files for later processing.
- `post_process_after_run`: automatically calls `run_deform_post_processing.py` after the multihit run finishes.

GUI-derived choices

The GUI enforces several derived choices before saving or running a config:

- `action_version` is inferred from the selected action CSV filename, for example `_v6_` becomes `v6`. If no version is present, the GUI uses `v1`.
- `use_jmak = true` writes `jmak_simulation_modes = "D H G"` and keeps the mean grain size field active.

- `use_jmak = false` writes `jmak_simulation_modes = "D H"` and clears the mean grain size field.
- Action filenames containing `bdm` set symmetric die motion when action filename rules are enabled.
- The stock diameter field is constrained to the supported inch values used by the current key/action workflow.

Die movement modes

`symmetric` writes MOVCTL blocks for both the top die and bottom die. The dies move toward and away from the workpiece in mirrored directions. This is the preferred mode when bending needs to be minimized.

`single_top_die` writes the normal top-die stroke while the bottom die stays stationary during the forming portion of the hit. Near the end, the bottom die retracts using `bottom_die_retract_speed_multiplier` and `bottom_die_retract_lead_steps`.

The same mode logic works for both `die_motion_axis = "z"` and `die_motion_axis = "y"`. Axis-specific interference directions, offsets, and MOVCTL direction vectors come from `die_positioning.py`.

Hit 0 behavior

Hit 0 prepares the starting state. It does not solve a forming hit.

The runner reads the base key file, optionally changes the material, optionally initializes the JMAK grain size, positions the die or dies flush with the workpiece, backs them out by `height`, writes the hit-0 DB, and exports the initial key/coordinate files.

Hit 1 through final hit

For every forming hit, the runner reads the current action row, builds a stroke plan, writes MOVCTL controls, writes a DEFORM DB, runs the DB, reads the solved DB, exports final outputs, and prepares the next-hit key if more hits remain.

Between hits, the workpiece can be repositioned and rotated according to the next action row. The dies are then re-seated around the current workpiece using the configured die axis and motion mode.

Material changes

Material changes are handled by `material_management.py`. The supported `material_source` values are:

- `system_library`: import a DEFORM material from the standard material library.
- `user_library`: import a material from a user material library folder.
- `key`: import material data from a key string or key file.
- `db`: import material data from a DEFORM database.
- `assign`: assign an existing material number to the target object.

The default target object is `Workpiece`, controlled by `material_object_name`.

Outputs

Each run creates a run folder under `data_path/run_name`. Each hit has its own subfolder named with the run name and hit number.

Expected outputs include:

- DEFORM DB files for each hit.
- Initial, final, and post-hit key files when `export_post_hit_keys` is true.
- Coordinate logs for Workpiece, Top Die, and Bottom Die when `export_coordinate_logs` is true.
- Load-stroke CSV files with columns `Step`, `Time`, `S`, `V`, `Fx`, `Fy`, `Fz` when `export_load_stroke` is true.
- `post_processed` CSV and JSON summaries when `post_process_after_run` is true.

Referenced code files

- `deform_config_gui.py`: user-facing configuration, action-file creation, DOE launch, post-processing, plotting, and manual access.
- `die_positioning.py`: die axis vectors, hit-0 positioning, and coordinate logs.
- `material_management.py`: programmatic DEFORM material import/assignment.
- `grain_initialization.py`: supplier initial grain-size key-string generation.
- `run_deform_post_processing.py`: per-run post-processing.
- `microstructure_post_processing.py`: GRAIN key parsing.